PCB Planarity Testing

Tanzi Emam

Franklin Noble

Brandon Spiller

Graph Theory Deep Dive

Discussion

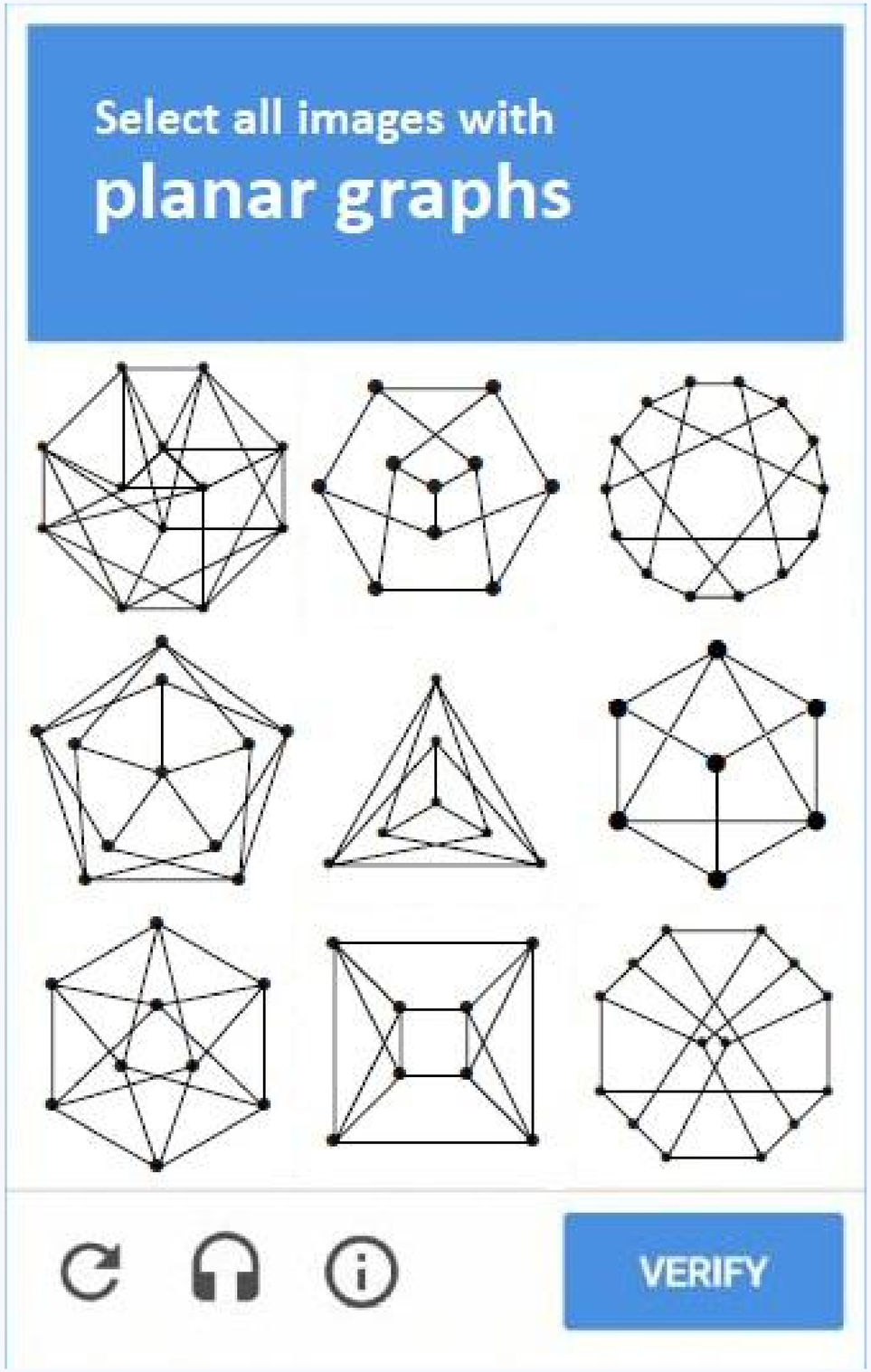
Electrical components on a PCB can be represented as a graph, where verticies are components and edges are traces (connections) between components. The number of layers that a certain schematic requires can be determined by computing the planarity of the schematic, where a nonplanar schematic means more than one layer is required. To determine a theoretical minimum number of layers required, an algorithm could iteratively take nonplanar subgraphs of a larger schematic, running planarity tests until a subgraph is planar. The total number of subgraph divisions required is then the minimum number of layers.

Algorithms

We ultimately used a Hamiltonian Cycle planarity detection algorithm, however we evaluated other programs such as DMP and K3,3/K5 algorithms. We discuss why we chose this algorithm and drawbacks/benefits of all the programs.

Result & Conclusion

Planarity detection is extremely useful to set base guidelines for the beginning of a PCB design by returning a theoretical minimum number of layers. Our implementation returns the minimum number of layers that would be required for a given electrical layout on a PCB by determining the planarity of the layout and continually making subgraphs of non-planar components.



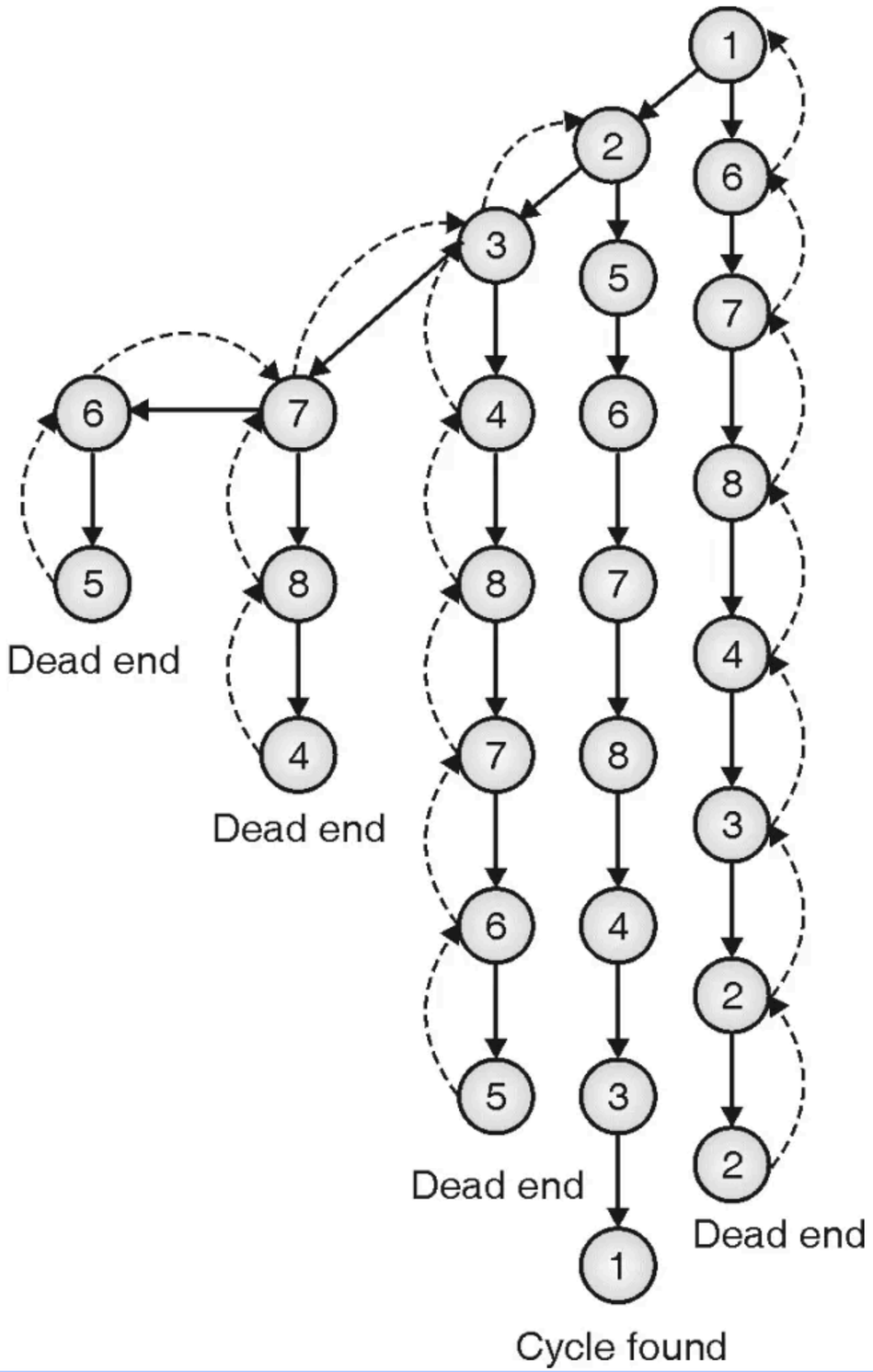
Finding a Hamiltonian Cycle Through Depth-First Search

Pseudocode pt1

- Select a vertex in the graph
- Visit a neighbor vertex and mark as ‘visited’, adding it to the current path
- If there are no further neighbors (path is a dead end), remove all verticies from ‘visited’ and return to previous vertex
- Repeat (recursion)
- If the length of the path is equal to the number of verticies, verify there is a edge back to the starting vertex. If so, a valid Hamiltonian Cycle is found

Discussion

- This is a polynomial time search algorithm
- We use this to detect if the planarity of graphs can be solved with our simple algorithm
- If no Hamiltonian Cycle is found, we cannot determine planarity of the PCB graph



Planarity Detection of a Hamiltonian Graph

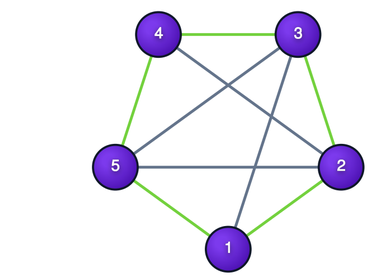
Pseudocode pt2

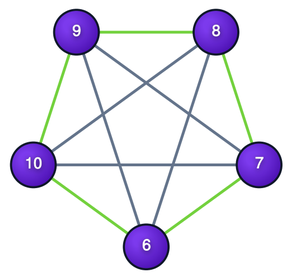
- Find a Hamiltonian Cycle.
- Draw the Hamiltonian Cycle as a polygon, and list the edges of the graph that are not part of the cycle.
- For each edge inside the polygon, label it as Inside (I) or Outside (O) attempting to avoid any intersections of edges with the same label.
- If label combinations are found where no edges of the same label (I or O) intersect, the graph is planar.
- Else, the graph is nonplanar.

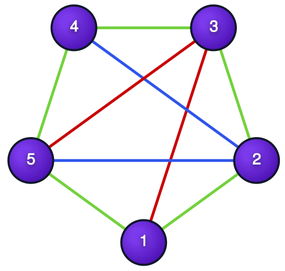
Discussion

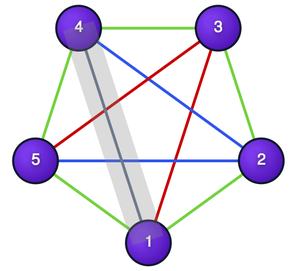
- This algorithm only works on graphs with Hamiltonian Cycles
- Nonlinear time complexity
- Other algorithms are significantly more complex and difficult to implement, but work in linear time and on any graph

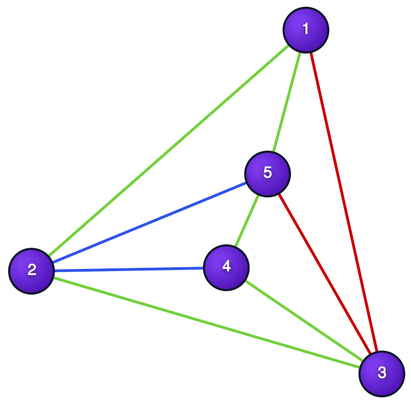
Example

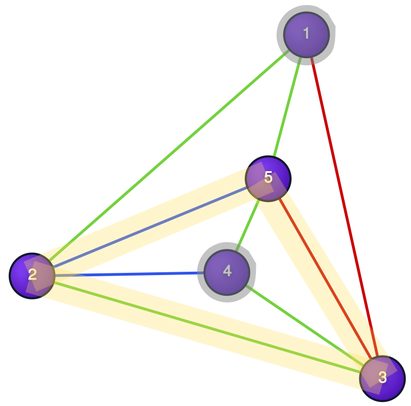












Planar

Non-Planar

Representing PCBs as Graphs

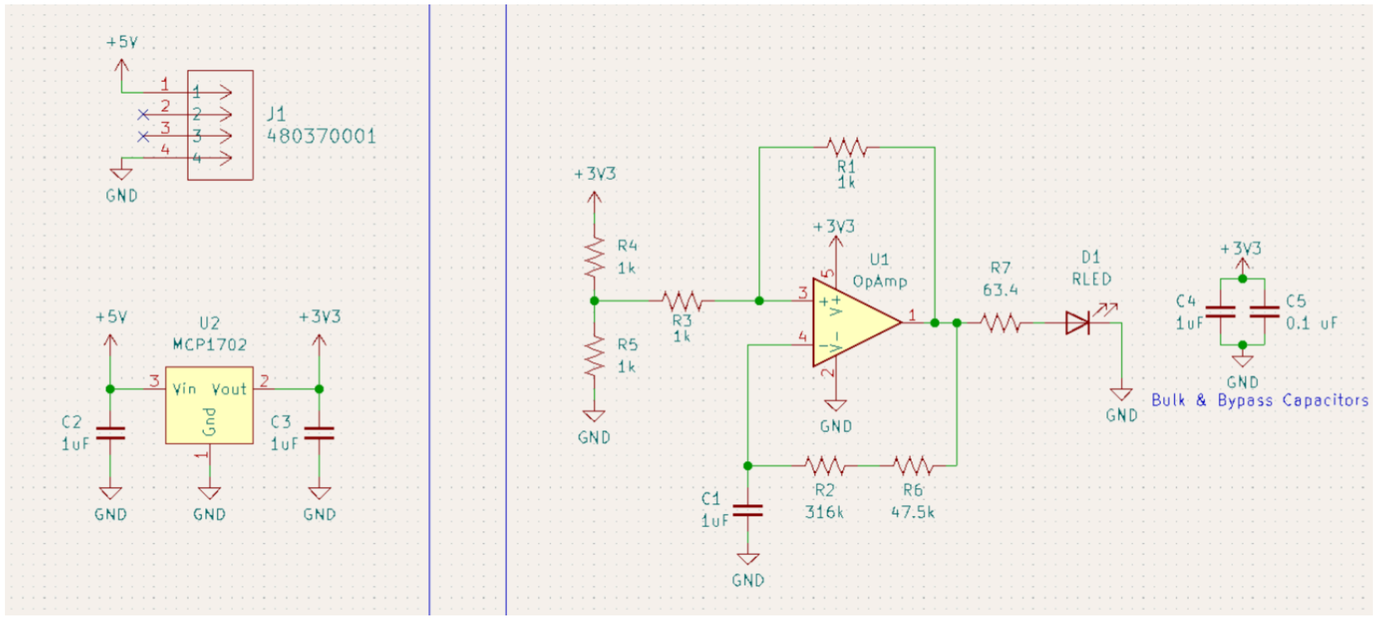
Pseudocode pt3

- In our code, when a graph returns as nonplanar it also returns the positions of the original vertices (representing where component pins are on the PCB) and the leftover edges that make it nonplanar.
- It takes the the leftover edges and the positions of the vertices and recursively passes it back through the process of finding a hamiltonian cycle, labeling edges Inside (I), Outside (O) or Leftover, and then repeats this recursion until it has all planar subgraphs. Each time it repeats the recursion it adds +1 to layer count.

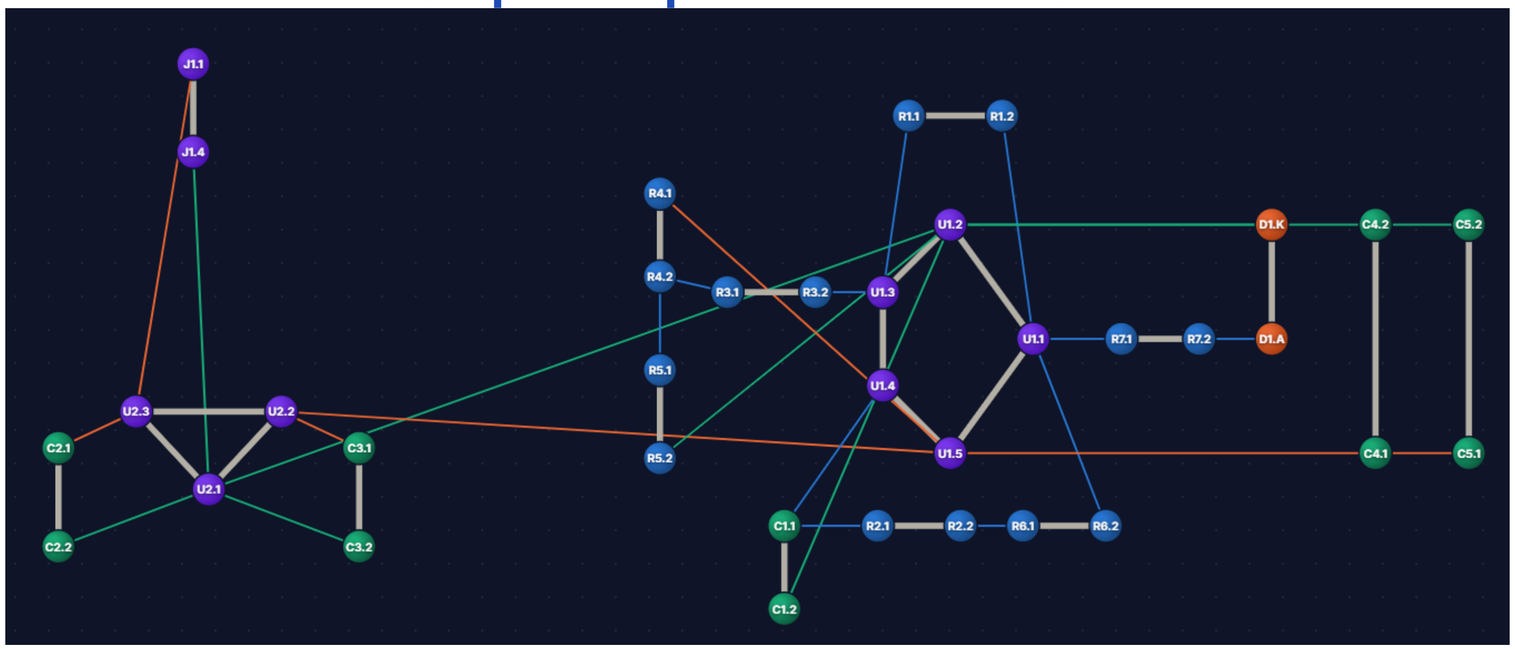
Discussion

We do not have limitations with what can be represented, as all connections in a PCB can be abstracted as vertices and edges. This algorithm that we have implemented will tell you the minimum number of layers necessary, but that may not be best practice for professional PCB design

KiCad Schematic



Graphical representation of PCB

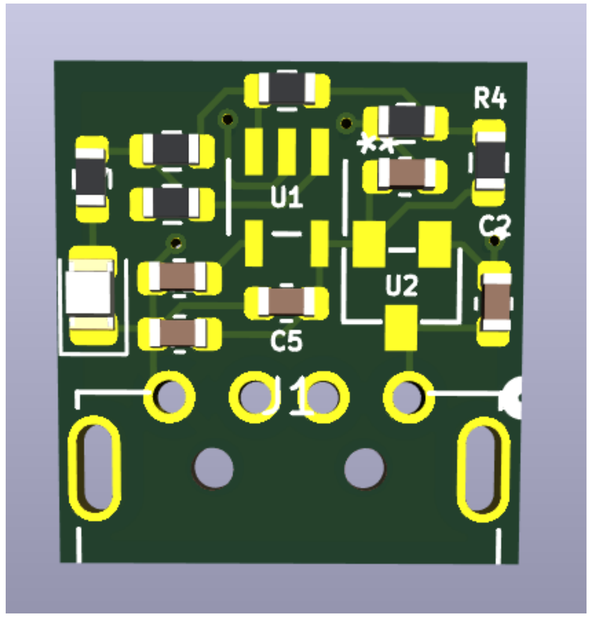


Representing PCBs as Graphs

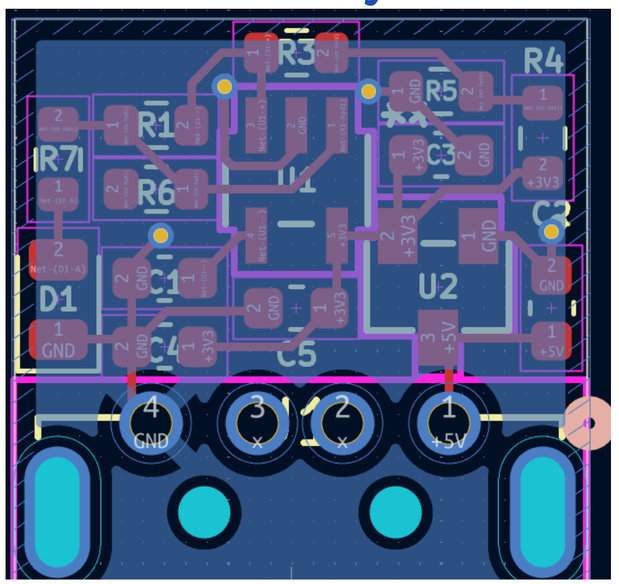
Assumptions for PCB Design

- On a 2-pin component, a trace can't pass between its two pads
- Traces on the same net conflict in our implementation, where on a PCB they would connect to eachother
- Large nets such GND are represented as traces, when they would normally be pours, this inflates the number of edge conflicts
- Our implementation doesn't take into account the boards physical dimensions into it's planarity test or layer counting.

PCB 3d Viewer Model



PCB Layout



Red is Layer 1
Blue is Layer 2

Terminal Output of our Python Implementation

```
python -u "c:\Users\bspiller\OneDrive - Olin College of Engineering\Desktop\Planar_PCBs.py"
Components: ['C1', 'C2', 'C3', 'C4', 'C5', 'D1', 'J1', 'R1', 'R2', 'R3', 'R4', 'R5', 'R6', 'R7', 'U1', 'U2']
Connections: [('C1', 'C2'), ('C1', 'R2'), ('C2', 'J1'), ('C2', 'C3'), ('C3', 'C4'), ('C4', 'C5'), ('C5', 'R4'), ('C5', 'D1'), ('D1', 'J1'), ('D1', 'R7'), ('J1', 'U2'), ('J1', 'R5'), ('
R1', 'R6'), ('R1', 'R3'), ('R2', 'R6'), ('R2', 'U1'), ('R3', 'R4'), ('R3', 'U1'), ('R4', 'U1'), ('R4', 'R5'), ('R5', 'U1'), ('R6', 'R7'), ('R7', 'U1'), ('U1', 'U2')]
Hamiltonian cycle: ['C1', 'C2', 'C3', 'C4', 'C5', 'D1', 'R7', 'R6', 'R1', 'R3', 'R4', 'R5', 'J1', 'U2', 'U1', 'R2']
Non-cycle edges: [('C2', 'J1'), ('C5', 'R4'), ('D1', 'J1'), ('R2', 'R6'), ('R3', 'U1'), ('R4', 'U1'), ('R5', 'U1'), ('R7', 'U1')]
Crossing pairs: [(('C2', 'J1'), ('R2', 'R6')), (('C2', 'J1'), ('R3', 'U1')), (('C2', 'J1'), ('R4', 'U1')), (('C2', 'J1'), ('R5', 'U1')), (('C2', 'J1'), ('R7', 'U1')), (('C5', 'R4'), ('
D1', 'J1')), (('C5', 'R4'), ('R2', 'R6')), (('C5', 'R4'), ('R3', 'U1')), (('C5', 'R4'), ('R7', 'U1')), (('D1', 'J1'), ('R2', 'R6')), (('D1', 'J1'), ('R3', 'U1')), (('D1', 'J1'), ('R4',
 'U1')), (('D1', 'J1'), ('R5', 'U1')), (('D1', 'J1'), ('R7', 'U1')), (('R2', 'R6'), ('R7', 'U1'))]
NONPLANAR: conflicting crossing constraints.
Conflicting edges: ('C2', 'J1') and ('R2', 'R6')
Conflicting edges: ('C2', 'J1') and ('R3', 'U1')
Conflicting edges: ('C2', 'J1') and ('R4', 'U1')
Conflicting edges: ('C2', 'J1') and ('R5', 'U1')
Conflicting edges: ('C2', 'J1') and ('R7', 'U1')
Conflicting edges: ('C5', 'R4') and ('D1', 'J1')
Conflicting edges: ('C5', 'R4') and ('R2', 'R6')
Conflicting edges: ('C5', 'R4') and ('R3', 'U1')
Conflicting edges: ('C5', 'R4') and ('R7', 'U1')
Conflicting edges: ('D1', 'J1') and ('R2', 'R6')
Conflicting edges: ('D1', 'J1') and ('R3', 'U1')
Conflicting edges: ('D1', 'J1') and ('R4', 'U1')
Conflicting edges: ('D1', 'J1') and ('R5', 'U1')
Conflicting edges: ('D1', 'J1') and ('R7', 'U1')
Conflicting edges: ('R2', 'R6') and ('R7', 'U1')
layer count is: 2
PS C:\Users\bspiller>
```

Alternative Planarity Detection Algorithms

Discussion

There are many algorithms for planarity detection, including those that work in linear time and on arbitrary graphs. To the side is a shortlist of various algorithms that we could have implemented instead.

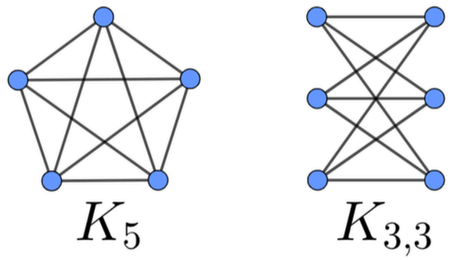
However, all these algorithms are far more complex than the Hamiltonian Cycle algorithm, and would require far more time and effort to implement than we had available to us for this project.

If we were to continue on this project, we would attempt to implement DMP, which is similar to the Hamiltonian Cycle algorithm, but works on all graphs, not just ones that have a Hamiltonian Cycle.

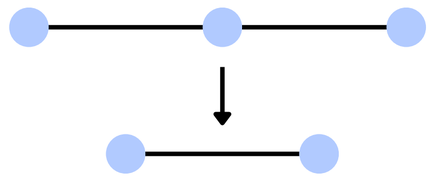
Example

Ex 1: K3 and K5 Detection

It has been mathematically proven that every nonplanar graph must include K3,3 (the complete 3-3 bipartite graph) or K5 (the complete graph with 5 vertices) as a subgraph or minor. Thus, a class of general planarity detecting algorithms looks for K3,3 or K5 within graphs. If they are not found, the graph is necessarily planar.

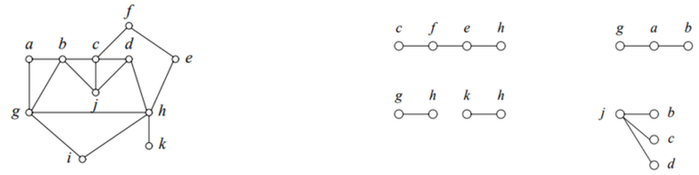


*A graph minor is a graph that can be created by combining some number of vertices connected by an edge:



Ex 2: DMP

The DMP algorithm is similar to the Hamiltonian Cycle algorithm. It first splits a graph into biconnected components, then for each one finds a small cycle, and progressively adds edges meeting specific criteria to create more faces of the graph. If it can't add an edge under these criteria, the graph is nonplanar.



Ex 3: Boyer-Myrvold

The Boyer-Myrvold algorithm is a linear-time algorithm that builds a graph by using edge addition. The algorithm represents a graph as a tree, and keeps track of the biconnected components it builds. When adding an edge combines biconnected components, thus creating a new face, the algorithm may need to 'flip' the component to ensure that all future edges are added only on the 'outside' face.

